Routed Experts Are Not Redundant: Training Dynamics and Branch Proportion in Small Mixture-of-Experts Models

AUTHOR NAMES
AFFILIATION
email@domain

***Abstract*—Small Mixture-of-Experts language models built with an always-on shared expert appear to under-use their routed experts: removing the entire routed branch costs little perplexity, while removing the single shared expert costs a great deal. We show that this asymmetry reflects optimization dynamics and architectural proportion rather than any property of the computation being performed. Training Mixture-of-Experts models of 58M to 251M parameters on a domain-labelled corpus, we find the imbalance is transient, the two branches equalizing by roughly 300M tokens, and we trace it to a gradient-exposure race in which the shared expert, updated on every token, outcompetes routed experts updated on a k/E fraction of them. The race is causally controllable: slowing the shared expert reallocates computation to the routed branch, and a handicap applied over only the first tenth of training is enough to make that reallocation permanent. Where the branches finally settle is governed not by the number of experts but by the ratio of their widths, which orders eleven configurations spanning eightfold in ratio and eightfold in expert count. Across that entire range held-out perplexity varies by less than 0.1, so the placement of feed-forward computation in a small Mixture-of-Experts model is close to a free design choice.**

***Index Terms*—mixture of experts, training dynamics, sparse models, language models, expert specialization**

I. Introduction

Mixture-of-Experts (MoE) layers replace a dense feed-forward block with a router that dispatches each token to a small number of experts. The DeepSeekMoE design [1] adds an always-on *shared* expert beside the *routed* ones, on the intuition that the shared expert absorbs computation common to all tokens and thereby frees the routed experts to specialize. Because small MoEs are inexpensive to train and every token can be labelled, they have become a common testbed for studying how experts divide work.

In this small-scale regime a striking observation recurs. Ablating the entire routed branch barely moves held-out perplexity, whereas ablating the lone shared expert degrades it severely. Read at face value, the measurement says the routed experts are nearly useless, and it invites two explanations: either small MoEs are over-parametrized on the routed side, or the shared expert actively suppresses the routed ones. Both explanations attribute the imbalance to the architecture.

We argue that neither is right, and that the imbalance says almost nothing about what the computation requires. It is instead the joint product of two things that have no obvious connection to the task: how quickly each branch is trained, and how wide each branch is. The apparent redundancy is transient, disappearing once the model has seen a few hundred million tokens. Its cause is a race for gradient signal that the shared expert wins by construction, and that a single learning-rate multiplier settles in either direction at no cost in perplexity. Applying that multiplier over the first tenth of training suffices; once the race is decided, the allocation persists after the handicap is removed. Finally, where the two branches come to rest is set by the ratio between their widths, and not, as the mechanism naively suggests, by the number of routed experts. Holding the routed branch identical down to its parameter shapes and varying only the shared expert's width moves the shared branch's share of ablation impact from 0.28 to 0.81.

Taken together these results reframe the shared/routed split as a near-neutral reparametrization. Held-out perplexity varies by less than 0.1 across configurations whose branch split spans 0.25 to 0.81, which means a practitioner may place the computation more or less where they wish. That freedom matters whenever the routed experts are themselves the object of interest, as they are in interpretability work, expert pruning, and modular editing, and it means that a diagnosis of expert redundancy drawn from an undertrained checkpoint is not merely imprecise but misleading.

II. Related Work

Conditional computation through a sparsely gated MoE layer was introduced by Shazeer et al. [2] and scaled by GShard [3] and Switch Transformer [4], the latter establishing the auxiliary load-balancing loss that we also use; ST-MoE [5] added the router z-loss for stability. Our backbone is a top-2 configuration in the style of Mixtral [6].

The always-on shared expert originates with DeepSeek-MoE [1] and carries through DeepSeek-V2 and V3 [7], [8], motivated by the argument that isolating common knowledge in a shared branch lets the routed experts specialize more sharply. That argument concerns what the branches represent. Our results concern how the branches are trained and proportioned, and they suggest that at small scale the second consideration dominates the first.

Observational studies of expert behaviour report that experts organize more by token identity and surface form than by

topic, both in Mixtral [6] and across training in OLMoE [9]. These studies characterize the outcome of training. We add a controlled causal probe and identify a training-dynamics cause for the small-scale imbalance that observational analysis alone cannot isolate.

III. Experimental Setup

a) Model: A decoder-only MoE implemented from scratch in PyTorch, using Multi-head Latent Attention [7], RoPE [10] and RMSNorm [11]. Each layer carries E routed experts and one shared expert with top-k routing. The reference configuration has $E = 8$, $k = 2$, 8 layers, hidden size 512, context 512 and 134M parameters; every expert, routed or shared, has intermediate width 1024. Section VI varies the two widths independently and writes them I per routed expert and I_{shared} for the shared one. Configurations range from 58M to 251M parameters. All runs use batch 64, sequence length 512, learning rate 3×10^{-4} with 1500 warmup steps, and an auxiliary load-balancing coefficient of 10^{-2}; 30 000 steps correspond to 983M tokens. Every experiment fits on one RTX A6000.

b) Data: Pre-training streams three token-labelled corpora: FineWeb-Edu [12] (60%), Cosmopedia-v2 [13] (25%) and OpenWebMath [14] (15%), tokenized with a 32k vocabulary and packed to 512 tokens. Held-out evaluation uses a skip-offset of the same streams rather than a fully disjoint split.

c) Measuring where the computation lives: We ablate a whole branch, either all routed experts or the shared expert, and record the resulting increase in held-out perplexity. Ablating a single expert is uninformative under top-k routing because the router simply reallocates its tokens to the survivors; only whole-branch ablation isolates a branch's contribution. We summarize the two measurements by the shared branch's share of total ablation impact,

$$s = \frac{\Delta_{\text{shared}}}{\Delta_{\text{shared}} + \Delta_{\text{routed}}}, \tag{1}$$

so that $s = 0.5$ denotes branches of equal importance and $s \to 1$ a shared-dominated model. Every trajectory we measured dips to a minimum and then rises slowly, so we report both the minimum reached and the value at 983M tokens. Seed-to-seed variation on s is approximately 0.02, an order of magnitude below the effects discussed below.

IV. Branch Importance Is a Function of Training Tokens

We train the reference model for 983M tokens with four seeds and measure branch importance at every checkpoint. Table I and Fig. 1 show the result.

At 98M tokens the shared expert accounts for 81% of the ablation impact and the routed experts do indeed look close to redundant. This is the regime in which many pilot studies operate, including our own earlier experiments. By 295M tokens the branches have equalized, and they remain balanced through 983M; routed specialization grows in parallel, with domain-conditioned normalized mutual information rising from 0.0008 to 0.017. All four seeds follow the same trajectory with tight error bars.

TABLE I
Branch ablation impact over training, four seeds, mean ± std.

tokens	PPL	remove shared	remove routed	s
98M	4.18	$+3.44 \pm 0.17$	$+0.82 \pm 0.03$	0.81
197M	3.63	$+2.37 \pm 0.17$	$+1.58 \pm 0.06$	0.60
295M	3.41	$+1.99 \pm 0.14$	$+1.98 \pm 0.06$	0.50
491M	3.20	$+1.99 \pm 0.16$	$+2.25 \pm 0.01$	0.47
983M	3.00	$+2.45 \pm 0.18$	$+2.48 \pm 0.11$	0.50

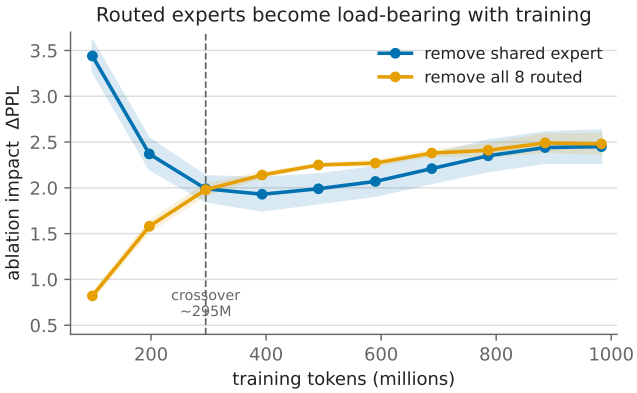


Fig. 1. Ablation impact of removing the shared expert against all eight routed experts, over a four-seed trajectory at constant batch size. Bands are ±1 standard deviation. The branches cross to equal importance by roughly 295M tokens.

The practical reading is narrow but firm. Branch importance is not a property of the architecture that can be measured once; it is a function of how much data the model has seen. Conclusions about routed redundancy drawn below a few hundred million tokens do not survive further training, and the effect is large enough that the sign of a comparison can change.

V. A Gradient-Exposure Race

Why should the shared expert lead at all? The asymmetry is present in the update rule before it is present in the weights. Writing the block output on token t as

$$y_t = s(x_t) + \sum_{e \in \text{topk}(t)} g_{t,e}\, f_e(x_t), \tag{2}$$

with s the shared expert and f_e the routed ones, a batch of N tokens delivers gradient from all N of them to the shared parameters but from only about Nk/E to any given routed expert. Under a common learning rate the shared expert therefore accumulates roughly E/k times more effective updates per parameter during the early phase, when gradients are large and the allocation of work is still plastic. The residual stream compounds this: it routes common computation to whichever submodule reduces loss fastest, which further starves the routed experts of useful gradient.

TABLE II
SHARED-EXPERT LEARNING-RATE MULTIPLIER, TWO SEEDS, MEASURED AT 98M TOKENS.

m	remove routed	remove shared	s	PPL
1.0 (baseline)	+0.85	+3.46	0.80	4.16
0.3	+1.94	+2.48	0.56	4.10
0.1	+4.46	+0.72	0.14	4.10

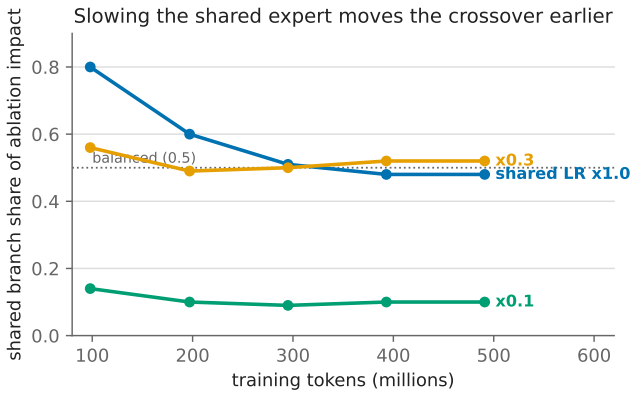


Fig. 2. Shared branch share against training tokens for three shared-expert learning-rate multipliers, two seeds. Slowing the shared expert moves the crossover earlier, dose-dependently, at unchanged perplexity.

This account makes two predictions. Reducing the shared expert's learning rate should let the routed experts capture computation earlier, and the crossover should scale with E/k. The first holds; the second does not, and Section VI is devoted to what replaces it.

A. The race can be rigged

We place the shared expert's parameters in their own optimizer group at $\mathrm{lr} \times m$ and leave everything else untouched. Table II reports the state at 98M tokens, deep in the regime where the baseline looks shared-dominated.

Slowing the shared expert makes the routed branch load-bearing far earlier: a fivefold increase in routed ablation impact at a fixed token budget, shifting the crossover from roughly 295M tokens to below 100M at $m = 0.3$ and to before training settles at $m = 0.1$. The effect is monotone and dose-dependent. It refutes the hypothesis that routed experts simply mature on their own schedule; they were being out-competed.

Held-out perplexity is unchanged across the three conditions, and if anything marginally better when the shared expert is slowed. The shared expert is therefore not necessary for the computation it performs in the baseline. The model places the work there because that is where it can be learned fastest, not because that is where it belongs.

B. The control holds at greater depth

A mechanism that depended on our particular depth would be of little interest, so we repeat the sweep on a model of twice the depth, 16 layers and 251M parameters. The control reproduces almost exactly. At 98M tokens the shared share falls from 0.82 to 0.64 to 0.21 across the same three multipliers, against 0.80, 0.56 and 0.14 at 8 layers, and perplexity is again unchanged. The right panel of Fig. 3 places the two depths side by side.

What depth does change is the timing of the unaided crossover, and it changes it in the direction that makes the phenomenon more treacherous rather than less. The 8-layer baseline crosses to equal importance at roughly 295M tokens and stays there. The 16-layer baseline never crosses within our budget: it reaches a minimum of 0.54 near 491M tokens and then drifts back up to 0.59 by 983M, as the left panel of Fig. 3 shows. Deeper models therefore remain in the shared-dominated regime for considerably longer, which is what the mechanism predicts, since more residual capacity gives the shared expert's head start more room to compound. It also means the warning of Section IV grows stronger with scale rather than weaker: the larger the model, the longer an observer will be able to look at it and conclude, wrongly, that its routed experts are redundant. We emphasize that this concerns the crossover reached without intervention; the following subsection shows the deeper model can be driven well past balance and kept there.

C. A temporary handicap is enough

If the split is decided by an early race, a handicap covering only that race should suffice, and the allocation should not revert once it is lifted. We test this on the 16-layer model by ramping m linearly from 0.1 back to 1.0 by a chosen step, after which every parameter trains at the base rate. Fig. 4 shows the outcome for four ramp endpoints.

The reallocation does not undo. Once the handicap is removed the share stays near where the handicap left it rather than returning to the baseline trajectory, and the shared branch re-asserts itself only slowly. The response is monotone in the duration of the handicap: the share at 983M tokens is 0.33, 0.26, 0.21 and 0.19 for handicaps lifted at 98M, 197M, 295M and 491M tokens respectively, against 0.59 with no handicap at all and 0.14 for a handicap never lifted. Final perplexities are 2.88, 2.88, 2.88 and 2.89, against 2.90 for the baseline and 2.91 when the handicap is never released.

Three features of that curve matter more than its monotonicity. A brief handicap already does much of the work: lifting it after 98M tokens, one tenth of the budget, reaches 0.33, which is more than half of the distance from the baseline to the never-released limit. The increments then shrink, by 0.07, 0.05 and 0.02 across the successive endpoints, but the never-released limit at 0.14 lies a further 0.05 below the longest ramp, so the response flattens without reaching a plateau anywhere in the range we swept. And releasing the handicap is not merely harmless but marginally preferable: every ramped condition ends at 2.88, better than both the untouched baseline and the permanently handicapped run, which is the only condition here that is worse than doing nothing. The knob is thus a schedule intervention rather than a standing tax on the shared expert, which is what a race-then-commit dynamic predicts,

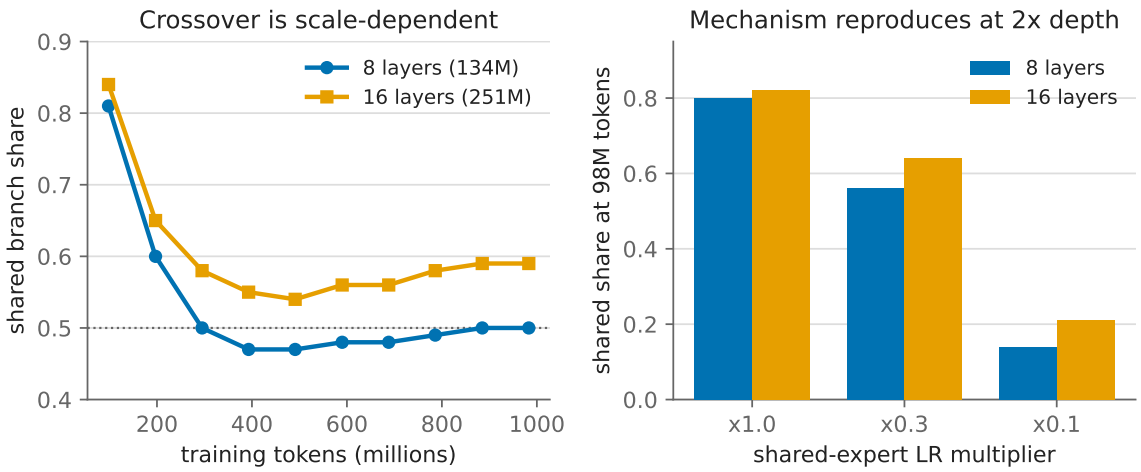


Fig. 3. Left: shared branch share against tokens for the unaided baseline at two depths. The 8-layer model crosses to equal importance; the 16-layer model approaches but does not, within 983M tokens. Right: the learning-rate control reproduces at twice the depth, lowering the share dose-dependently at both scales.

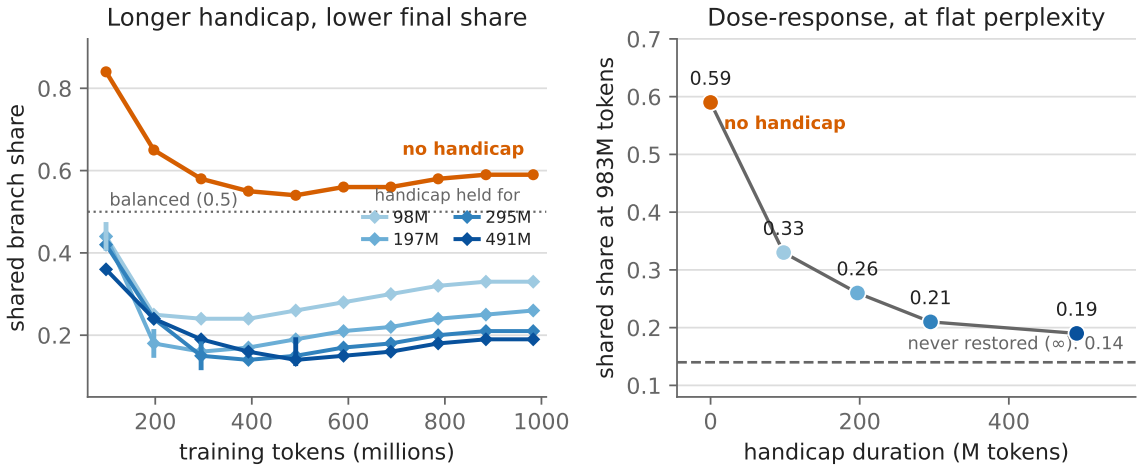


Fig. 4. Left: shared branch share against training tokens at 16 layers, for four handicap durations; the tick marks the point at which each handicap was lifted. Right: the resulting dose-response, final share against how long the handicap was held. Perplexity is flat across the whole series.

since what wins the early race is what the residual stream subsequently builds upon.

We replicated the 295M endpoint with a second seed and obtained identical values. The remaining three endpoints are single-seed, and the sweep covers one depth and one handicap strength, so we report the shape of the dependence rather than a functional law.

VI. The Width Ratio Sets Where the Split Lands

The mechanism explains why the shared expert leads and shows that the lead can be reversed. It does not say where the split comes to rest, and the natural prediction from the gradient-exposure argument, that the resting point should follow E/k, turns out to be wrong. Across eleven 8-layer configurations trained to 983M tokens, expert count is nearly irrelevant and the ratio of the two branches' widths orders every run.

TABLE III
TWO DISENTANGLING CONTRASTS, BOTH AT E=8, k=2. VALUES ARE THE MINIMUM SHARED SHARE REACHED OVER THE TRAJECTORY.

Varying routed width, $I_{\text{shared}} = 1024$			
I	256	1024	2048
$s_{\min}$	0.79	0.47	0.37
parameters	58.2M	133.7M	234.4M
Varying shared width, $I = 1024$; routed branch identical			
I_{shared}	512	1024	4096
$s_{\min}$	0.28	0.47	0.81
parameters	127.4M	133.7M	171.5M

Table III isolates the two candidates. Both contrasts hold

$E = 8$ and $k = 2$ fixed, so expert count, routing and per-expert gradient share are constant throughout. The second contrast is the decisive one: with I pinned at 1024 the routed branch is identical across the three runs, the same number of experts of the same width receiving the same routed capacity $k{\cdot}I = 2048$, and yet the split moves by 0.53. Whatever governs the resting point, it is not the absolute capacity of the routed branch.

What the two contrasts share is the ratio I/I_{shared}. Grouping every configuration by that ratio yields bands that do not overlap, as Fig. 5 shows: shared shares of 0.72–0.81 at ratio 0.25, 0.63 at ratio 0.5, 0.47–0.56 at ratio 1.0 and 0.25–0.37 at ratio 2.0. The ordering survives an eightfold range in ratio, an eightfold range in expert count and a fourfold range in parameter count.

Expert count does not explain the residual spread within a band, and it does not even act in a consistent direction: at ratio 2.0 more experts raise the share, from 0.25 at $E = 4$ to 0.37 at $E = 8$, while at ratio 0.25 more experts lower it, from 0.79 at $E = 8$ to 0.72 at $E = 32$. A reader might still suspect that the sweep confounds expert count with capacity, since a naive sweep over E at fixed per-expert width grows the routed branch as it grows E. Holding the product $E \cdot I$ at 8192 removes that confound and equalizes parameter counts; perplexity is then flat across $E \in \{4, 8, 16, 32\}$ at 2.99, 3.00, 3.00 and 3.02, while the share continues to track the ratio that the same manipulation necessarily varies. Expert count is close to a free parameter for both quantities in this regime.

The two results compose rather than compete. The gradient race decides which branch arrives first; the relative width of the branches decides how much computation each can absorb, and therefore where the allocation settles once the race is over. Both are reparametrizations rather than levers on quality: across the seven configurations between 127M and 234M parameters the minimum shared share spans 0.25 to 0.81 while final perplexity spans only 2.95 to 3.02.

VII. Does Reallocation Buy Specialization?

The motivation for wanting routed experts engaged is usually that they are the object of study, so it matters whether experts forced to carry more computation also organize their work more sharply. We measure gate-weighted normalized mutual information between expert assignment and the domain label of each token, comparing at matched training steps and, where possible, at identical routed architecture. Fig. 6 collects the result, and it is not the simple one.

The width lever delivers. Holding $E = 8$ and $I = 1024$ so that the routed branch is identical across runs and varying only I_{shared}, domain NMI at 983M tokens rises from 0.0103 to 0.0174 to 0.0226 as the shared share falls from 0.82 to 0.50 to 0.34. That is a factor of 2.2 in specialization obtained without touching the routed experts at all, and without cost in perplexity.

The learning-rate lever does not, or not reliably. At 16 layers, measured at 491M tokens where the baseline remains shared-dominated, slowing the shared expert raises NMI monotonically from 0.0108 to 0.0135 to 0.0151. At 8 layers, where the baseline has already crossed to balance by that point, the same intervention moves NMI the other way, from 0.0191 at $m = 1.0$ to 0.0175 at $m = 0.1$. The dose-response series of Section V is similarly equivocal: NMI increases with handicap duration among the ramped runs but the unhandicapped baseline falls inside their range rather than below it.

The pattern these disagreements share is that specialization tracks how much the routed branch has genuinely been trained, which rises as it takes over computation but flattens near balance and can fall once the shared branch is starved past it. Engagement and specialization are therefore related but distinct axes, and the distinction is practical: an experimenter who wants routed experts that are better objects of interpretability should change the branch widths rather than the learning rate, even though both move the split by comparable amounts. We note that domain labelling is one specialization axis among several, that our 16-layer learning-rate runs are single-seed, and that we deliberately exclude comparisons across expert count here because NMI grows mechanically with the number of experts.

VIII. Discussion

The picture that emerges is that a small MoE's division of feed-forward labour is only weakly determined by the task. Two quantities that a practitioner sets freely, the relative learning rate of the two branches and their relative width, move the split across almost its entire attainable range while leaving held-out perplexity within 0.1. The shared expert is not doing something the routed experts could not do; it is doing what it got to first, and it keeps as much of the work as its width allows.

Three consequences follow. The first is diagnostic. Expert redundancy cannot be assessed from a single early checkpoint, because the measurement at 98M tokens and the measurement at 983M tokens disagree, and deeper models remain in the misleading regime longer. Branch importance should be reported as a trajectory. The second is a matter of experimental design: our own E/k prediction survived a sweep over expert count only because that sweep let routed capacity ride along with the variable under study, and it failed as soon as capacity and parameter count were matched. Sweeps over architecture should fix what they are not testing. The third is prescriptive, and Section VII makes it sharper than we expected. A modest learning-rate handicap on the shared expert engages the routed branch much sooner at no measured cost, and need only be applied over the early part of training; but engagement bought that way does not reliably come with sharper specialization. If the routed experts are wanted merely *used*, either lever serves. If they are wanted *legible*, the width ratio is the one that works.

Several limits bound these claims, and they concern generality rather than internal validity. All models here fall between 58M and 251M parameters and see at most 983M tokens, on a single data mixture and a single hardware configuration; whether the crossover and the two controls persist at production scale is untested and is the obvious next question. The

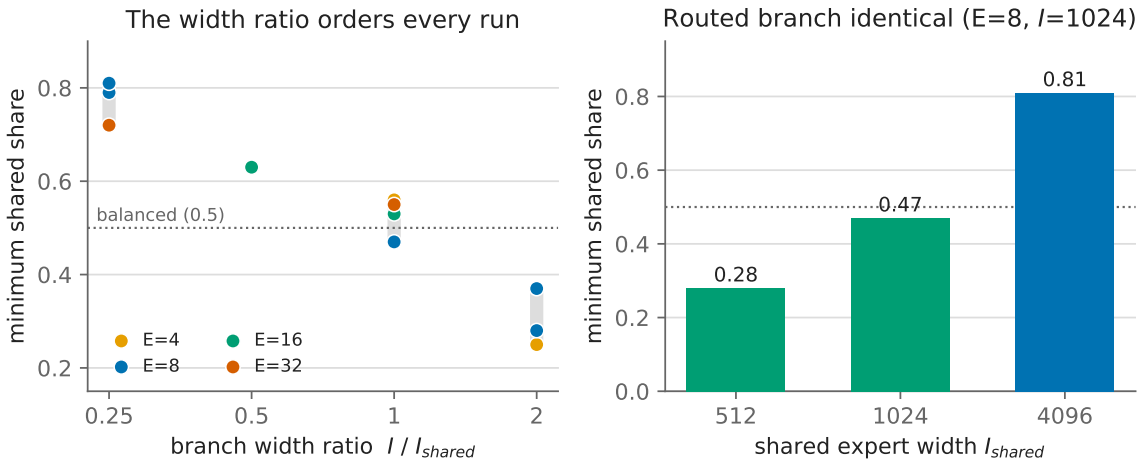


Fig. 5. Left: minimum shared share against the branch width ratio, one point per configuration, coloured by expert count; the bands at each ratio do not overlap. Right: the decisive contrast, in which the routed branch is identical across all three runs and only the shared expert's width changes.

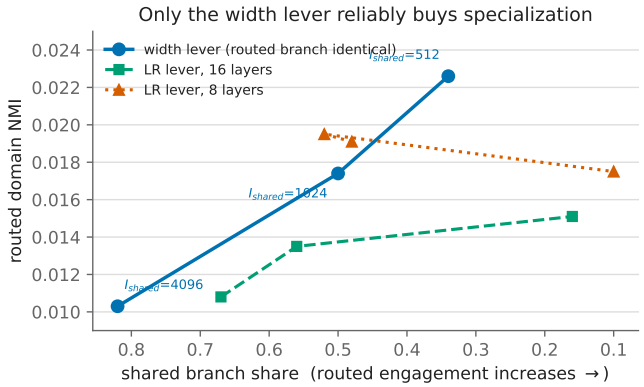


Fig. 6. Routed domain specialization against branch split, with routed engagement increasing to the right. The width lever, whose three runs share a byte-identical routed branch, more than doubles specialization. The learning-rate lever helps at 16 layers, where the baseline is still shared-dominated, and hurts at 8 layers, where it is not.

width-ratio result establishes an ordering over an eightfold range at one depth, not a functional form, and we deliberately do not fit one. The dose-response over handicap duration is replicated at a single endpoint, and its remaining points rest on one seed each; the effect sizes there exceed the measured seed noise by a comfortable margin, but the shape of the saturation deserves a denser sweep. Finally, our evidence is perplexity, ablation impact and routing statistics. At this scale standard reasoning and question-answering benchmarks sit at or near chance, so we cannot say whether a reallocated split changes downstream behaviour; that is the measurement a practitioner would ultimately want, and answering it at all requires a larger model rather than a further experiment on this one.

IX. Conclusion

Routed experts in small Mixture-of-Experts models are not redundant. They are out-competed early in training by a shared expert that receives gradient from every token, and the outcome of that competition is settled by two hyperparameters rather than by the demands of the data. Slowing the shared expert reverses the result, permanently, even when applied only over the first tenth of training; the relative width of the two branches then fixes where the split comes to rest, spanning shared shares from 0.25 to 0.81 while perplexity moves by less than 0.1. Where a small Mixture-of-Experts model puts its feed-forward computation is, to a good approximation, something the designer chooses rather than something the model discovers.

Reproducibility

The model, training code and analysis toolkit, including branch ablation, routing capture and the trajectory drivers, are released with this paper; every experiment runs on a single GPU. Two implementation defects found and fixed during the study, a missing causal mask in the absorbed attention inference path and a broken CUDA reduction in the load-balancing loss, are documented in the version history.

References

[1] D. Dai, C. Deng, C. Zhao *et al.*, "DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models," *arXiv preprint arXiv:2401.06066*, 2024.

[2] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations (ICLR)*, 2017.

[3] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "GShard: Scaling giant models with conditional computation and automatic sharding," in *International Conference on Learning Representations (ICLR)*, 2021.

[4] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research (JMLR)*, 2022.

[5] B. Zoph, I. Bello, S. Kumar, N. Du, Y. Huang, J. Dean, N. Shazeer, and W. Fedus, "ST-MoE: Designing stable and transferable sparse expert models," *arXiv preprint arXiv:2202.08906*, 2022.
[6] A. Q. Jiang, A. Sablayrolles, A. Roux *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.
[7] DeepSeek-AI, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.
[8] ——, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.
[9] N. Muennighoff, L. Soldaini, D. Groeneveld *et al.*, "OLMoE: Open mixture-of-experts language models," *arXiv preprint arXiv:2409.02060*, 2024.
[10] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021.
[11] B. Zhang and R. Sennrich, "Root mean square layer normalization," *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
[12] G. Penedo, H. Kydlíček, A. Lozhkov *et al.*, "The FineWeb datasets: Decanting the web for the finest text data at scale," *arXiv preprint arXiv:2406.17557*, 2024.
[13] L. Ben Allal, A. Lozhkov, G. Penedo *et al.*, "Cosmopedia: How to create large-scale synthetic data for pre-training," *Hugging Face Blog*, 2024.
[14] K. Paster, M. D. Santos, Z. Azerbayev, and J. Ba, "OpenWebMath: An open dataset of high-quality mathematical web text," *arXiv preprint arXiv:2310.06786*, 2023.